

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 1

Operating Systems

Topic: The OS as a Resource Manager

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

CONTENTS

INTRODUCTION 3
1 THEORETICAL BACKGROUND 4
1.1 Operating Systems as Resource Managers 4
1.2 Linux Interfaces Used in This Laboratory 4
2 SYSTEM IDENTIFICATION 5
2.1 Working Directory and User Identity 5
2.2 Kernel Information 5
2.3 System Uptime 5
3 FILES AND DIRECTORIES 7
3.1 Navigate and Inspect 7
3.2 Create, Copy, Rename, and Remove Files 8
3.3 Inspect Ownership and Permissions 9
4 PROCESSES 10
4.1 List Running Processes 10
4.2 Start, Find, and Stop a Background Process 11
4.3 Inspect a Process Through /proc 11
5 MEMORY 13
5.1 Inspect System Memory and Swap 13
5.2 Inspect Resident Memory of a Process 13
6 DEVICES AND STORAGE 15
6.1 Inspect Mounted Filesystems 15
6.2 Inspect Block Devices 15
6.3 Measure the Laboratory Folder 16
6.4 Inspect Device Files 16
CONCLUSION 18
REFERENCES 19

INTRODUCTION

This laboratory work looks at the operating system as a resource manager that sits between applications and the hardware. The practical part makes four core responsibilities visible from the terminal: file management, process management, memory management, and device and storage management.

The report follows the order of the lab handout. Each exercise gives the commands that were run, the output captured from the terminal session, and a short comment that answers the corresponding “Observe” question in my own words.

The objectives are to identify the user, kernel and uptime of the system; to inspect files, directories, ownership and permissions; to observe processes and the /proc interface; to examine RAM and swap; and to inspect mounted filesystems and device files, linking each observation to the resource that the OS manages.

The work was done on Ubuntu running under WSL2 (Windows Subsystem for Linux) instead of a stand-alone virtual machine, with kernel 6.18.33.2-microsoft-standard-WSL2 on x86_64. The user account is userdalvgame, the host name is WIN-07CNHC94OG2, and the working directory is /home/userdalvgame/os-lab1. Because WSL2 runs Linux inside a lightweight Windows-managed virtual machine, a few outputs (virtual disks, the /mnt/c mount, a small process list) differ from those of a full Linux VM; these differences are pointed out where they matter. The sessions were recorded with the script command.

1 THEORETICAL BACKGROUND

1.1 Operating Systems as Resource Managers

An operating system lets many programs share one computer safely. It gives names, owners and permissions to stored data through files and directories; it creates, isolates and schedules processes; it gives every process its own view of virtual memory while sharing the physical RAM; and it hides disks, terminals and other hardware behind uniform, file-like interfaces. Programs never have to manage the hardware directly.

These mechanisms can be observed from outside the kernel. File metadata shows ownership and permissions, process listings and the /proc pseudo-filesystem expose process state, memory commands summarise RAM and swap, and filesystem and device commands show how storage is presented to users.

1.2 Linux Interfaces Used in This Laboratory

The shell is the user's way to ask the OS for services and to display the resulting state. ls, ps, free, df and lsblk each summarise a different managed resource. The /proc directory is generated by the kernel on demand and does not exist on disk, while /dev contains device nodes through which programs reach devices with ordinary file operations.

All numbers in this report (process identifiers, memory totals, device names) belong to my own WSL2 Ubuntu instance and will differ on other machines.

2 SYSTEM IDENTIFICATION

Before examining the four resource-management areas, the working directory is created and the account, kernel and system uptime are identified.

2.1 Working Directory and User Identity

```
mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami
```

The evidence for this exercise is shown in Figure 2.1.

```
userdalvgame@WIN-07CNHC94OG2:/mnt/c/Users/user$ mkdir -p ~/os-lab1
userdalvgame@WIN-07CNHC94OG2:/mnt/c/Users/user$ cd ~/os-lab1
userdalvgame@WIN-07CNHC94OG2:~/os-lab1$ whoami
userdalvgame
```

Figure 2.1: Creation of the laboratory directory and identification of the current user

Comment and observation

The directory /home/userdalvgame/os-lab1 was created and whoami reported userdalvgame as the active user. Every file created during the lab is therefore owned by this account (UID 1000, confirmed later in /proc) unless the ownership is changed. The prompt also shows the host name WIN-07CNHC94OG2, which WSL inherits from the Windows computer name.

2.2 Kernel Information

```
uname -a
```

The evidence for this exercise is shown in Figure 2.2.

```
$ uname -a
Linux WIN-07CNHC94OG2 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18
21:54:43 UTC 2026 x86_64 GNU/Linux
```

Figure 2.2: Kernel information reported by uname

Comment and observation

The machine runs Linux kernel 6.18.33.2-microsoft-standard-WSL2, built on 18 June 2026, on the x86_64 architecture. SMP means the kernel can use several processors and PREEMPT_DYNAMIC means its preemption behaviour can be chosen at boot. The suffix microsoft-standard-WSL2 shows that this is the Linux kernel that Microsoft ships for WSL2: the real Linux kernel runs in a small virtual machine, and the Ubuntu programs run on top of it.

2.3 System Uptime

```
uptime
```

The evidence for this exercise is shown in Figure 2.3.

```
$ uptime
 16:52:25 up 2 min,  1 user,  load average: 0.00, 0.00, 0.00
```

Figure 2.3: System uptime and load information

Comment and observation

At 16:52:25 the system had been running for only 2 minutes, which matches the moment WSL was started (the session log begins at 16:50). One user was logged in. The load averages (1, 5 and 15 minutes) were all 0.00, i.e. practically no process was waiting for the CPU – the machine was idle.

3 FILES AND DIRECTORIES

This part explores the filesystem tree, creates and removes files, and compares file permissions before and after chmod.

3.1 Navigate and Inspect

```
pwd                 # current directory
ls -la /            # root directory
ls -la ~            # home directory
cd /etc && ls | head
```

The evidence for this exercise is shown in Figures 3.1–3.2.

```
$ pwd
/home/userdalvgame/os-lab1
$ ls -la /
total 2840
drwxr-xr-x  19 root root    4096 Sep 28 16:50 .
drwxr-xr-x  19 root root    4096 Sep 28 16:50 ..
lrwxrwxrwx   1 root root       7 Apr 20 11:46 bin -> usr/bin
drwxr-xr-x   2 root root    4096 Apr 20 11:46 boot
drwxr-xr-x  15 root root    3840 Sep 28 16:50 dev
drwxr-xr-x  91 root root    4096 Sep 28 16:50 etc
drwxr-xr-x   3 root root    4096 Sep 16 14:34 home
-rwxr-xr-x   1 root root 2837280 Sep 11 08:14 init
lrwxrwxrwx   1 root root       7 Apr 20 11:46 lib -> usr/lib
lrwxrwxrwx   1 root root       9 Apr 20 11:46 lib64 -> usr/lib64
drwx------   2 root root   16384 Sep 16 14:31 lost+found
drwxr-xr-x   2 root root    4096 Aug 27 17:10 media
drwxr-xr-x   6 root root    4096 Sep 16 14:31 mnt
drwxr-xr-x   2 root root    4096 Aug 27 17:10 opt
dr-xr-xr-x 296 root root       0 Sep 28 16:50 proc
drwx------   4 root root    4096 Sep 25 23:52 root
drwxr-xr-x  22 root root     580 Sep 28 16:50 run
lrwxrwxrwx   1 root root       8 Apr 20 11:46 sbin -> usr/sbin
drwxr-xr-x   2 root root    4096 Sep 16 14:31 snap
drwxr-xr-x   2 root root    4096 Aug 27 17:10 srv
dr-xr-xr-x  13 root root       0 Sep 28 16:50 sys
drwxrwxrwt   7 root root     140 Sep 28 16:55 tmp
drwxr-xr-x  12 root root    4096 Aug 27 17:10 usr
drwxr-xr-x  13 root root    4096 Sep 16 14:31 var
```

Figure 3.1: The current directory and the root directory of the filesystem tree

```
$ ls -la ~
total 60
drwxr-x--- 10 userdalvgame userdalvgame 4096 Sep 28 16:52 .
drwxr-xr-x  3 root        root        4096 Sep 16 14:34 ..
-rw-------  1 userdalvgame userdalvgame 3517 Sep 27 02:38 .bash_history
-rw-r--r--  1 userdalvgame userdalvgame  220 Sep 16 14:34 .bash_logout
-rw-r--r--  1 userdalvgame userdalvgame 3797 Sep 26 21:13 .bashrc
drwxr-x---  6 userdalvgame userdalvgame 4096 Sep 27 00:14 .cache
drwxr-x---  5 userdalvgame userdalvgame 4096 Sep 26 21:13 .config
-rw-r--r--  1 userdalvgame userdalvgame  833 Sep 26 21:13 .profile
drwxr-xr-x  2 userdalvgame userdalvgame 4096 Sep 28 16:52 os-lab1
  ... (other hidden folders and project directories omitted) ...
$ cd /etc && ls | head
PackageKit
X11
adduser.conf
alternatives
apparmor
apparmor.d
apport
apt
bash.bashrc
bash_completion
```

Figure 3.2: The home directory and the first entries of /etc

Comment and observation

The current working directory was /home/userdalvgame/os-lab1. Two directories directly under / are /etc, which holds system-wide configuration files (for example adduser.conf, bash.bashrc and the apt settings), and /home, which holds the personal directories of the users, including /home/userdalvgame.

Almost everything under / is owned by root and writable only by root. Several entries (bin, lib, lib64, sbin) are symbolic links into /usr. proc and sys are virtual directories produced by the kernel (size 0), tmp has the sticky bit (drwxrwxrwt) so every user can create files but only delete their own, and root is drwx------ so only root may enter. The home directory is drwxr-x---: only its owner may write to it, and other users are denied access. A WSL-specific entry is /mnt, where the Windows drives are mounted.

3.2 Create, Copy, Rename, and Remove Files

```
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

The evidence for this exercise is shown in Figure 3.3.

```
$ cd ~/os-lab1
$ mkdir -p demo
$ cd demo
$ echo "hello operating systems" > note.txt
$ cp note.txt copy.txt
$ mv copy.txt renamed.txt
$ ls -l
total 8
-rw-r--r-- 1 userdalvgame userdalvgame 24 Sep 28 16:58 note.txt
-rw-r--r-- 1 userdalvgame userdalvgame 24 Sep 28 16:58 renamed.txt
$ rm renamed.txt
$ ls -l
total 4
-rw-r--r-- 1 userdalvgame userdalvgame 24 Sep 28 16:58 note.txt
```

Figure 3.3: Creation, copying, renaming, listing and removal of files

Comment and observation

The sequence created note.txt with the text “hello operating systems”, copied it to copy.txt and renamed the copy to renamed.txt. The first listing shows two files of 24 bytes each (23 characters plus the newline added by echo). Both are owned by user userdalvgame and belong to group userdalvgame, which is the private group Ubuntu creates for each user. After rm renamed.txt the second listing shows only note.txt, and the total fell from 8 to 4 blocks, which is the disk space the removed file used.

3.3 Inspect Ownership and Permissions

```
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
```

The evidence for this exercise is shown in Figure 3.4.

```
$ ls -l note.txt
-rw-r--r-- 1 userdalvgame userdalvgame 24 Sep 28 16:58 note.txt
$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 userdalvgame userdalvgame 24 Sep 28 16:58 note.txt
$ chmod 644 note.txt
$ ls -l note.txt
-rw-r--r-- 1 userdalvgame userdalvgame 24 Sep 28 16:58 note.txt
```

Figure 3.4: Permission changes for note.txt

Comment and observation

After chmod 600 the mode string is -rw-------. The first character - marks a regular file (d would be a directory, l a link). The next three characters rw- give the owner read and write permission but not execute; the following --- denies everything to the group and the last --- denies everything to other users. In octal, 600 means 6 (4+2 = read+write) for the owner and 0 for the others. chmod 644 restored -rw-r--r--: the owner can read and write, while group and others can only read. The OS checks these bits every time a process opens the file.

4 PROCESSES

This part lists running processes, starts and stops a background process, and inspects the information exported by the kernel through /proc.

4.1 List Running Processes

```
ps aux | head
ps aux | wc -l     # approximate process count
ps -p 1 -o pid,comm,args
top                # press q to quit
```

The evidence for this exercise is shown in Figures 4.1–4.2.

```
$ ps aux | head
USER   PID %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root     1  0.2  0.1  24312 15416 ?     Ss   16:50   0:01 /sbin/init
root     2  0.0  0.0   3180  2204 hvc0  Sl+  16:50   0:00 /init
root     7  0.0  0.0   3180  2040 hvc0  Sl+  16:50   0:00 plan9 --control-socket 7 ...
root    47  0.0  0.1  50400 17276 ?     S<s  16:50   0:00 /usr/lib/systemd/systemd-journald
systemd+ 79  0.0  0.1  22416 14428 ?     Ss   16:50   0:00 /usr/lib/systemd/systemd-resolved
root    85  0.0  0.1  35448 12324 ?     Ss   16:50   0:00 /usr/lib/systemd/systemd-udevd
root   114  0.0  0.0   4472  3156 ?     Ss   16:50   0:00 /usr/sbin/cron -f -P
$ ps aux | wc -l
29
$ ps -p 1 -o pid,comm,args
    PID COMMAND         COMMAND
      1 systemd         /sbin/init
```

Figure 4.1: Process listing, process count and identification of PID 1

```
top - 17:00:41 up 10 min,  1 user,  load average: 0.10, 0.04, 0.01
Tasks:  27 total,   1 running,  26 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.4 id,  0.0 wa,  0.0 hi,  0.6 si,  0.0 st
MiB Mem :   9813.7 total,   9152.7 free,    609.3 used,    211.6 buff/cache
MiB Swap:   3072.0 total,   3072.0 free,      0.0 used.   9204.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   24312  15416  11608 S   0.0   0.2   0:01.49 systemd
      2 root      20   0    3180   2204   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
     47 root      19  -1   50400  17284  15996 S   0.0   0.2   0:00.54 systemd-journal
    114 root      20   0    4472   3156   2908 S   0.0   0.0   0:00.00 cron
    318 userdal+  20   0    6268   5600   3928 S   0.0   0.1   0:00.04 bash
```

Figure 4.2: Live process view (top, first screen)

Comment and observation

Process number 1 has PID 1. ps aux shows its command as /sbin/init, and ps -p 1 -o comm reveals the real program name: systemd. It is the init and service manager; the kernel starts it first and every other user-space process descends from it. It starts and supervises the services (journald, resolved, udevd, cron, ...) that appear next in the list. Two WSL-specific processes are visible as well: PID 2, /init, is the WSL init process, and PID 7 is a plan9 server used for file sharing with Windows.

ps aux | wc -l printed 29 lines: one header and therefore 28 processes. top reported 27 tasks in its first screen, 1 running (top itself) and 26 sleeping, with 99.4% of the CPU idle and no stopped or zombie processes. The small difference between 28 and 27 is expected because the two commands look at the system at different moments and because the process list changes

continuously. So an idle WSL Ubuntu has roughly 27–29 processes, far fewer than a full desktop system. The columns of top worth reading are PID, USER, state (S = sleeping, R = running), %CPU, %MEM, accumulated CPU time and command.

4.2 Start, Find, and Stop a Background Process

```
sleep 300 &            # note the printed PID
jobs
ps -ef | grep '[s]leep 300'
kill <PID>
```

The evidence for this exercise is shown in Figure 4.3.

```
$ sleep 300 &
[1] 2762
$ PID=$!
$ echo "$PID"
2762
$ jobs
[1]+  Running                 sleep 300 &
$ ps -ef | grep '[s]leep 300'
userdal+    2762     515  0 17:01 pts/2    00:00:00 sleep 300
$ kill "$PID"
$ jobs
[1]+  Terminated              sleep 300
```

Figure 4.3: Creation, discovery and termination of a background sleep process

Comment and observation

The shell started sleep 300 in the background as job [1] with PID 2762. jobs listed it as Running. ps -ef then found it among all processes, showing its parent PID 515 (the interactive shell) and the terminal pts/2. The pattern '[s]leep 300' is a small trick: the regular expression matches the text “sleep 300”, but grep’s own command line contains “[s]leep 300”, which does not match, so grep does not list itself. After kill 2762 the OS sent the process the default termination signal, and the next jobs call reported job [1] as Terminated.

4.3 Inspect a Process Through /proc

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
```

The evidence for this exercise is shown in Figures 4.4–4.5.

```
$ ls /proc/$PID/   (PID = 3011)
arch_status  coredump_filter  gid_map            map_files   ns             personality  smaps
task
attr         cpuset           io                 maps        numa_maps      projid_map   smaps_rollup
timens_offsets
auxv         cwd              ksm_merging_pages  mem         oom_adj        root         stack
timers
cgroup       environ          ksm_stat           mountinfo   oom_score      sched        stat
timerslack_ns
clear_refs   exe              latency            mounts      oom_score_adj  schedstat    statm
uid_map
cmdline      fd               limits             mountstats  pagemap        sessionid    status
wchan
comm         fdinfo           loginuid           net         patch_state    setgroups    syscall
```

Figure 4.4: Entries that the kernel exposes for one process in /proc

```
$ head -20 /proc/$PID/status
Name:  sleep
Umask:  0022
State:  S (sleeping)
Tgid:  3011
Ngid:  0
Pid:  3011
PPid:  515
TracerPid:  0
Uid:  1000  1000  1000  1000
Gid:  1000  1000  1000  1000
FDSize:  256
Groups:  4 24 27 30 46 100 986 1000
NStgid:  3011
NSpid:  3011
NSpgid:  3011
NSsid:  515
Kthread:  0
VmPeak:     16116 kB
VmSize:     16116 kB
VmLck:         0 kB
$ grep '^State:' /proc/$PID/status
State:  S (sleeping)
```

Figure 4.5: Kernel information for the sleep process read from /proc/<pid>/status

Comment and observation

For PID 3011 the State: line says S (sleeping). The process is waiting for its timer to expire and is not using the CPU. The same file shows the parent PID (515, the shell), the real and effective user and group IDs (all 1000, my account), the number of open-file slots (FDSize 256), and memory values such as VmPeak and VmSize of 16116 kB. None of these files exists on disk: the kernel builds them when they are read, so /proc is a file-like window into the kernel's own process table.

5 MEMORY

This part compares the system-wide memory totals with the resident memory of a single sleeping process.

5.1 Inspect System Memory and Swap

```
free -h                   # RAM and swap totals
cat /proc/meminfo | head -6
swapon --show
```

The evidence for this exercise is shown in Figure 5.1.

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           9.6Gi       613Mi       8.9Gi       3.5Mi       212Mi       9.0Gi
Swap:          3.0Gi          0B       3.0Gi
$ head -6 /proc/meminfo
MemTotal:       10049188 kB
MemFree:         9377156 kB
MemAvailable:    9431100 kB
Buffers:            4200 kB
Cached:           188400 kB
SwapCached:            0 kB
$ swapon --show
NAME     TYPE      SIZE USED PRIO
/dev/sdc partition   3G   0B   -2
```

Figure 5.1: System-wide RAM and swap information

Comment and observation

The VM has 9.6 GiB of RAM in total (MemTotal: 10049188 kB), of which 8.9 GiB is free and 9.0 GiB is available; only 613 MiB is used and 212 MiB is buffer/cache. The same numbers appear in top (9813.7 MiB = 10049188 kB / 1024). Swap is disk space the OS can use to store memory pages that are not needed right now, or when RAM runs short. 3.0 GiB of swap is configured (the block device /dev/sdc) and none of it was in use (0 B), which is normal for a machine with so much free RAM. free -h rounds values to human-readable units, while /proc/meminfo gives the exact values in kilobytes.

5.2 Inspect Resident Memory of a Process

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status    # resident memory
kill $PID
```

The evidence for this exercise is shown in Figure 5.2.

```
$ sleep 300 &
[1] 540
$ PID=$!
$ echo "Process PID: $PID"
Process PID: 540
$ grep VmRSS /proc/$PID/status
VmRSS:      7696 kB
$ kill "$PID"
$ jobs
[1]+  Terminated                sleep 300
```

Figure 5.2: Resident memory used by a sleep process (second terminal session)

Comment and observation

The bare sleep process (PID 540, measured in a second terminal session, hence the low PID) had a resident set of 7696 kB, about 7.5 MiB. This is more than I expected for a program that does nothing: its resident pages include the program's own code and stack plus the shared libraries it uses, and VmRSS counts every resident page that is mapped, even pages that are shared with other processes. VmRSS is the memory that is really in RAM, in contrast to VmSize (16116 kB in section 4.3), which is the whole virtual address space the process may use.

6 DEVICES AND STORAGE

This part inspects mounted filesystems, block devices, the size of the lab folder and device files exposed through /dev.

6.1 Inspect Mounted Filesystems

```
df -h                 # mounted filesystems and free space
mount | head
findmnt /
```

The evidence for this exercise is shown in Figures 6.1–6.2.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            4.8G     0  4.8G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            4.8G  4.0K  4.8G   1% /mnt/wsl
drivers         166G  161G  4.4G  98% /usr/lib/wsl/drivers
/dev/sdd       1007G   13G  944G   2% /
none            4.8G   40K  4.8G   1% /mnt/wslg
rootfs          4.8G  2.8M  4.8G   1% /init
none            4.8G  532K  4.8G   1% /run
C:\             166G  161G  4.4G  98% /mnt/c
D:\             789G  705G   84G  90% /mnt/d
tmpfs           4.8G     0  4.8G   0% /tmp
tmpfs           982M   12K  982M   1% /run/user/1000
  ... (other small virtual mounts omitted) ...
```

Figure 6.1: Mounted filesystems and available storage space (df -h)

```
$ mount | head
none on /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2 type overlay (rw,nosuid,nodev,...)
none on /mnt/wsl type tmpfs (rw,relatime)
drivers on /usr/lib/wsl/drivers type 9p (ro,nosuid,nodev,noatime,...)
/dev/sdd on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
none on /mnt/wslg type tmpfs (rw,relatime)
/dev/sdd on /mnt/wslg/distro type ext4 (ro,relatime,discard,errors=remount-ro,data=ordered)
rootfs on /init type rootfs (ro,size=5018836k,nr_inodes=1254709)
none on /dev type devtmpfs (rw,nosuid,relatime,size=5018836k,nr_inodes=1254709,mode=755)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,noatime)
$ findmnt /
TARGET SOURCE   FSTYPE OPTIONS
/      /dev/sdd ext4   rw,relatime,discard,errors=remount-ro,data=ordered
```

Figure 6.2: Mount table (mount | head) and the root mount (findmnt /)

Comment and observation

The root filesystem / is mounted from the device /dev/sdd and has type ext4. df -h reports a size of 1007 GB, 13 GB used, 944 GB available and 2% utilisation. df summarises capacity and usage per mount, whereas mount and findmnt show the source device, filesystem type and mount options. The output also shows how WSL joins two worlds: the Windows drives C:\ and D:\ are mounted at /mnt/c and /mnt/d through the 9p network-style filesystem, and the C: drive is 98% full (only 4.4 GB free), which is worth keeping in mind. Virtual filesystems such as tmpfs, devtmpfs, sysfs and proc are mounted too; they live in memory or are produced by the kernel and take no space on a disk.

6.2 Inspect Block Devices

```
lsblk
```

The evidence for this exercise is shown in Figure 6.3.

```
$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.4M  1 disk
sdc    8:32   0     3G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /mnt/wslg/distro
                               /
```

Figure 6.3: Block devices and their mount points

Comment and observation

lsblk lists four virtual disks. sdd (1 TB maximum size) holds the Ubuntu root filesystem and is mounted at / (and read-only at /mnt/wslg/distro); sdc (3 GB) is the swap device seen in section 5.1; sda and sdb are small read-only disks (RO = 1), most likely attached by WSL for its own system files. All four are virtual disks provided by the hypervisor, not physical drives, which is why a disk can be declared as 1 TB while only 13 GB of it is in use.

6.3 Measure the Laboratory Folder

```
du -sh ~/os-lab1
```

The evidence for this exercise is shown in Figure 6.4.

```
$ du -sh ~/os-lab1
12K   /home/userdalvgame/os-lab1
```

Figure 6.4: Disk space used by the laboratory working folder

Comment and observation

du -sh reports 12 KiB for the lab folder. The folder contains only the demo directory with the 24-byte note.txt, but du counts allocated disk blocks, not bytes: the two directories and the file each occupy at least one 4 KiB block (3 × 4 KiB = 12 KiB). It will grow when screenshots and report files are added.

6.4 Inspect Device Files

```
ls -l /dev | head
```

The evidence for this exercise is shown in Figure 6.5.

```
$ ls -l /dev | head
total 0
crw-r--r-- 1 root root     10, 235 Sep 28 16:50 autofs
drwxr-xr-x 2 root root        600 Sep 28 16:50 block
drwxr-xr-x 2 root root        120 Sep 28 16:50 bsg
crw------- 1 root root     10, 234 Sep 28 16:50 btrfs-control
drwxr-xr-x 2 root root       2860 Sep 28 16:50 char
crw------- 1 root root      5,   1 Sep 28 16:50 console
lrwxrwxrwx 1 root root         11 Sep 28 16:50 core -> /proc/kcore
crw------- 1 root root     10, 260 Sep 28 16:50 cpu_dma_latency
crw------- 1 root root     10, 203 Sep 28 16:50 cuse
```

Figure 6.5: Examples of device files exposed through /dev

Comment and observation

One example is /dev/console. The leading c marks a character device and 5, 1 are its major and minor numbers, which tell the kernel which driver handles it; it stands for the system console, the text input and output of the machine. Another is core, a link to /proc/kcore, a kernel interface presented as a file. “Everything is a file” means that Linux presents files, directories, disks, terminals and even kernel data through the same file interface, with names in one tree and the same permission bits, so programs can use open, read and write on all of them.

CONCLUSION

The OS managed files through directories, owners and permission bits, which I observed with ls -l and chmod. It managed processes through identifiers, states, parents and termination, observed with ps, top, kill and /proc.

Memory management was visible through free -h and /proc/meminfo, which show how RAM is split into used, free and cached memory and how swap supplements it, while VmRSS showed the physical memory resident for one process.

Devices and storage were observed with df, mount, findmnt, lsblk and ls -l /dev, which show how disks are presented as block devices, mounted into one tree, and how hardware and kernel interfaces appear as files. Doing the lab under WSL2 also showed that the same kernel concepts hold when Linux runs inside a lightweight virtual machine, with Windows drives appearing as just another mount.

REFERENCES

[1] Operating Systems Lab 1 handout. The OS as a Resource Manager. FCIM / FAF / UTM, 2026–2027.

[2] Linux manual pages: ps(1), top(1), proc(5), chmod(1), df(1), lsblk(8).